Payment Gateway Merchant Compliance API

Version: 1.0 | **Last Updated:** October 2026 | **Service:** gateway-merchant

Table of Contents

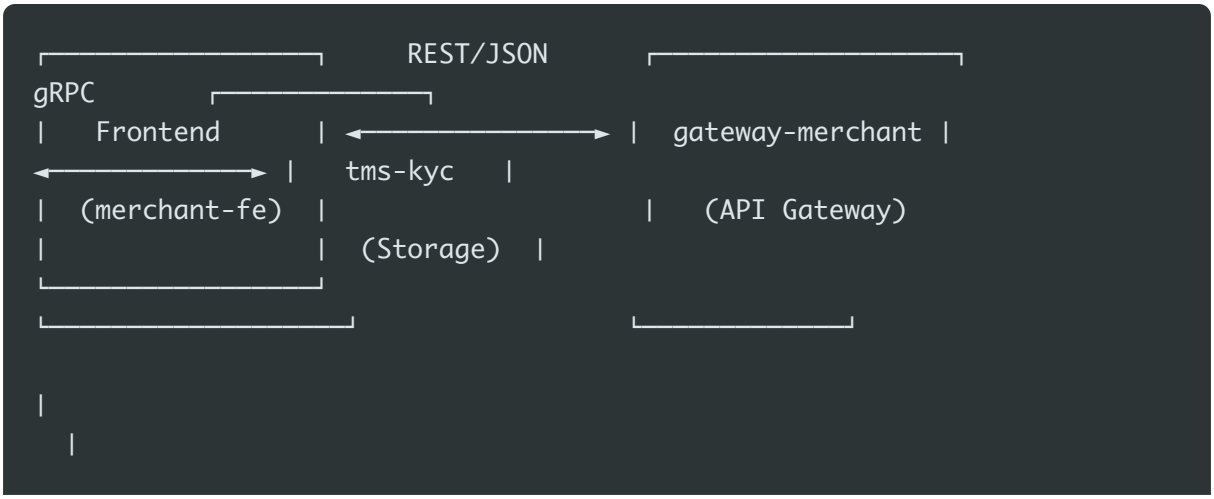
1. Overview
2. Authentication
3. Idempotency (Preventing Duplicates)
4. Optimistic Concurrency Control
5. Business Types
6. API Endpoints Reference
7. Starter Business Flow
8. Registered Business Flow
9. Document Upload Flow
10. Error Handling

1. Overview

The Payment Gateway Merchant Compliance API enables merchants to complete their KYC (Know Your Customer) requirements for payment gateway access. The API supports two business types:

- **Starter Business:** For unregistered businesses/sole proprietors (single owner)
- **Registered Business:** For CAC-registered companies (multiple shareholders)

Architecture



```
        |              Direct Upload (Presigned
URL)                                  |

└──────────────────────────────────────────────────────────────────────────
────┘
                                        AWS S3
```

Base URLs

Environment	URL
Production	`https://api.merchant.irpay.ng`
Staging	`https://api.merchant.staging.irpay.ng`

2. Authentication

All endpoints (except `/compliance/schema`) require Bearer token authentication.

Required Headers

```
Authorization: Bearer <access_token>
Content-Type: application/json
```

Obtaining a Token

```
POST /auth/login
Content-Type: application/json

{
  "email": "merchant@example.com",
  "password": "secure_password"
}

Response:
{
  "success": true,
  "data": {
    "access_token": "eyJhbGciOiJIUzI1NiIs...",
    "expires_in": 3600
```

```
  }
}
```

3. Idempotency (Preventing Duplicates)

What is Idempotency?

Idempotency ensures that if a request fails (network timeout, server error) and you retry it with the **same idempotency key**, the server will:

- Return the same response as the first successful attempt
- NOT create duplicate records or perform the action twice

> **Critical:** This is essential for financial applications where duplicate operations could cause serious problems.

How to Use

All mutating endpoints (POST, PUT, PATCH, DELETE) require an `Idempotency-Key` header:

```
Idempotency-Key: 550e8400-e29b-41d4-a716-446655440000
```

Rules for Idempotency Keys

Rule	Description
Format	UUID v4 (e.g., `550e8400-e29b-41d4-a716-446655440000`)
Uniqueness	Each distinct operation needs a unique key
TTL	Keys expire 24 hours after first use
Retry	Same key + same request = returns cached response
Conflict	Same key + different request = HTTP 409 error

When to Generate vs. Reuse Keys

Scenario	Action
User clicks "Save" first time	**Generate NEW key**
Request fails (network error)	**REUSE same key** for retry
Request succeeds	**DISCARD key**
User modifies data and clicks Save again	**Generate NEW key**

JavaScript Example

```
import { v4 as uuidv4 } from 'uuid';

let currentKey = null;  // Store key until operation succeeds

async function saveProfile(data) {
  // Generate key only on first attempt
  if (!currentKey) {
    currentKey = uuidv4();
  }

  const response = await fetch('/compliance/profile', {
    method: 'PUT',
    headers: {
      'Authorization': `Bearer ${token}`,
      'Idempotency-Key': currentKey,
      'X-Expected-Version': version,
      'Content-Type': 'application/json'
    },
    body: JSON.stringify(data)
  });

  if (response.ok) {
    currentKey = null;  // Success! Clear key for next operation
  }
  // On failure, key is retained for retry

  return response.json();
}
```

4. Optimistic Concurrency Control

What is It?

Prevents conflicts when multiple browser tabs or devices edit the same compliance record simultaneously.

How It Works

1. **GET** compliance → response includes `version: 5`
2. **PUT** with header `X-Expected-Version: 5`
3. If current version is still 5 → update succeeds, version becomes 6
4. If current version changed → HTTP 409 Conflict (refresh and retry)

Required Header

```
X-Expected-Version: 5
```

5. Business Types

Starter Business

For sole proprietors and unregistered small businesses.

Steps:

1. Profile Details
2. Contact Information
3. Owner Details (single owner with ID + proof of address)
4. Bank Account
5. Service Agreement

Documents Required:

- Owner ID document (National ID, NIN, Passport, or Driver's License)
- Owner proof of address (Utility bill ≤ 3 months old)

Registered Business

For CAC-registered companies (LLC, PLC, Business Name).

Steps:

1. Business Details
2. Registration Details (RC number, TIN)
3. Shareholders (1-10 people, each with ID + proof of address)
4. Bank Account
5. Service Agreement

Documents Required:

- Business proof of address
- Per shareholder: ID document + proof of address

6. API Endpoints Reference

Public Endpoint

Method	Endpoint	Description
GET	`/compliance/schema?business_type=X`	Get form schema (no auth required)

Compliance Endpoints

Method	Endpoint	Description
POST	`/compliance/start`	Start new compliance
GET	`/compliance`	Get current compliance state
PUT	`/compliance/profile`	Save profile/business details
PUT	`/compliance/contact`	Save contact information
PUT	`/compliance/registration`	Save registration (registered only)
PUT	`/compliance/owners`	Save owners/shareholders
PUT	`/compliance/bank-accounts`	Save bank accounts

POST	`/compliance/submit`	Submit for review

Document Endpoints

Method	Endpoint	Description
POST	`/compliance/documents/uploads`	Get presigned upload URL
POST	`/compliance/documents/{id}/complete`	Confirm upload complete
GET	`/compliance/documents/{id}`	Get document info
DELETE	`/compliance/documents/{id}`	Delete document

7. Starter Business Flow

Step 1: Get Schema

```
GET /compliance/schema?business_type=starter

Response: Form schema with fields for each step
```

Step 2: Start Compliance

```
POST /compliance/start
Headers:
  Authorization: Bearer <token>
  Idempotency-Key: <uuid-1>

Response:
{
  "compliance_id": 100,
  "status": "draft",
  "version": 1
}
```

Step 3: Save Profile

```
PUT /compliance/profile
Headers:
  Authorization: Bearer <token>
  Idempotency-Key: <uuid-2>
  X-Expected-Version: 1

Body:
{
  "country": "NG",
  "industry": "retail",
  "category": "fashion",
  "trading_name": "ACME Store",
  "sales_volume": "100000-500000",
```

```
  "business_type": "starter",
  "business_description": "Fashion retail business"
}

Response: { "version": 2, "profile_complete": true }
```

Step 4: Save Contact

```
PUT /compliance/contact
Headers:
  Idempotency-Key: <uuid-3>
  X-Expected-Version: 2

Body:
{
  "email": "contact@acme.com",
  "support_email": "support@acme.com",
  "dispute_email": "support@acme.com",
  "same_as_support_email": true,
  "phone": "+2348012345678",
  "address": "123 Victoria Island, Lagos",
  "lga": "eti-osa",
  "online_links": [
    { "platform": "website", "url": "https://acme.com" }
  ]
}

Response: { "version": 3, "contact_complete": true }
```

Step 5: Save Owner (Without Documents First)

```
PUT /compliance/owners
Headers:
  Idempotency-Key: <uuid-4>
  X-Expected-Version: 3

Body:
{
  "owners": [{
    "full_name": "John Doe",
    "date_of_birth": "1985-03-15",
    "nationality": "Nigerian",
```

```
    "role": "owner",
    "id_document_type": "international_passport",
    "address": "456 Ikoyi, Lagos",
    "lga": "eti-osa",
    "same_as_business_address": false
  }]
}

Response:
{
  "version": 4,
  "owners_complete": false,    ← Documents still missing!
  "owners": [
    { "id": 42, "full_name": "John Doe", ... }
  ]
}
```

Important: Note the owner `"id": 42` in the response. You'll need this when updating with document IDs.

Step 6: Upload Owner ID Document

6a. Request presigned URL:

```
POST /compliance/documents/uploads
Headers:
  Idempotency-Key: <uuid-5>

Body:
{
  "purpose": "identity",
  "file_name": "passport.pdf",
  "content_type": "application/pdf",
  "size_bytes": 524288
}

Response:
{
  "document_id": "doc_id_001",
  "upload_url": "https://s3.amazonaws.com/...",
  "method": "PUT",
```

```
  "headers": { "Content-Type": "application/pdf" }
}
```

> **Note:** For starter business, `"identity"` and `"owner_address"` purposes do NOT require `owner_id` because there's only one owner.

6b. Upload file directly to S3:

```
PUT https://s3.amazonaws.com/...
Content-Type: application/pdf
Body: <binary file data>

Response: 200 OK
```

6c. Confirm upload complete:

```
POST /compliance/documents/doc_id_001/complete
Headers:
  Idempotency-Key: <uuid-6>

Response: { "document_id": "doc_id_001", "status": "uploaded" }
```

Step 7: Upload Owner Proof of Address

Same 3-step pattern as Step 6:

```
// 7a. Request URL
POST /compliance/documents/uploads
{ "purpose": "owner_address", "file_name": "utility-bill.pdf", ... }
→ { "document_id": "doc_addr_001", "upload_url": "..." }

// 7b. Upload to S3
PUT <upload_url> with file

// 7c. Confirm complete
POST /compliance/documents/doc_addr_001/complete
```

Step 8: Update Owner with Document IDs

```
PUT /compliance/owners
Headers:
  Idempotency-Key: <uuid-7>
  X-Expected-Version: 4

Body:
{
  "owners": [{
    "id": 42,
    "full_name": "John Doe",
    "date_of_birth": "1985-03-15",
    "nationality": "Nigerian",
    "role": "owner",
    "id_document_type": "international_passport",
    "id_document_id": "doc_id_001",
    "proof_of_address_document_id": "doc_addr_001",
    "address": "456 Ikoyi, Lagos",
    "lga": "eti-osa",
    "same_as_business_address": false
  }]
}

Response: { "version": 5, "owners_complete": true }
```

Step 9: Save Bank Account

```
PUT /compliance/bank-accounts
Headers:
  Idempotency-Key: <uuid-8>
  X-Expected-Version: 5

Body:
{
  "bank_accounts": [{
    "bank_code": "044",
    "account_number": "0123456789",
    "account_name": "John Doe"
  }]
}
```

```
Response: { "version": 6, "bank_accounts_complete": true }
```

Step 10: Submit for Review

```
POST /compliance/submit
Headers:
  Idempotency-Key: <uuid-9>
  X-Expected-Version: 6

Body:
{
  "agreement_accepted": true
}

Response: { "version": 7, "status": "submitted" }
```

> **Starter Business Flow Complete!**

8. Registered Business Flow

Key Difference: For registered businesses with multiple shareholders, documents use **auto-linking** via `owner_id`:

1. Save shareholders first → get their IDs
2. Upload documents WITH `owner_id` → backend auto-links them
3. No need to re-save shareholders with document IDs!

Steps 1-2: Get Schema & Start Compliance

```
GET /compliance/schema?business_type=registered

POST /compliance/start
→ { "compliance_id": 200, "version": 1 }
```

Step 3: Save Business Details

```
PUT /compliance/profile
Headers:
  Idempotency-Key: <uuid>
  X-Expected-Version: 1

Body:
{
  "country": "NG",
  "industry": "technology",
  "category": "fintech",
  "trading_name": "TechCorp Limited",
  "sales_volume": "1000000-5000000",
  "business_type": "registered",
  "business_description": "Financial technology solutions"
}
```

```
Response: { "version": 2, "profile_complete": true }
```

Step 4: Save Contact with Business Address Document

4a. Upload business address document:

```
POST /compliance/documents/uploads
{ "purpose": "business_address", "file_name": "office-
lease.pdf", ... }
→ { "document_id": "doc_biz_addr" }

PUT <s3_url> (upload file)
POST /compliance/documents/doc_biz_addr/complete
```

4b. Save contact with document ID:

```
PUT /compliance/contact
Headers:
  Idempotency-Key: <uuid>
  X-Expected-Version: 2

Body:
{
  "email": "info@techcorp.ng",
  "support_email": "support@techcorp.ng",
  "dispute_email": "disputes@techcorp.ng",
  "phone": "+2349012345678",
  "address": "Plot 123 Tech Hub, Lekki",
  "lga": "eti-osa",
  "business_address_document_id": "doc_biz_addr",
  "online_links": [
    { "platform": "website", "url": "https://techcorp.ng" }
  ]
}

Response: { "version": 3, "contact_complete": true }
```

Step 5: Save Registration Details

```
PUT /compliance/registration
Headers:
  Idempotency-Key: <uuid>
  X-Expected-Version: 3

Body:
{
  "registration_number": "RC1234567",
  "tax_identification_number": "12345678-0001"
}

Response: { "version": 4, "registration_complete": true }
```

Step 6: Save Shareholders (Without Documents)

```
PUT /compliance/owners
Headers:
  Idempotency-Key: <uuid>
  X-Expected-Version: 4

Body:
{
  "owners": [
    {
      "full_name": "Adaora Okafor",
      "date_of_birth": "1980-05-20",
      "nationality": "Nigerian",
      "role": "director",
      "id_document_type": "international_passport",
      "address": "12 Executive Estate, Ikoyi",
      "lga": "eti-osa"
    },
    {
      "full_name": "Chidi Eze",
      "date_of_birth": "1975-11-08",
      "nationality": "Nigerian",
      "role": "shareholder",
      "id_document_type": "national_id",
      "address": "45 Business District, VI",
      "lga": "eti-osa"
    }
  ]
```

```
}

Response:
{
  "version": 5,
  "owners_complete": false,
  "owners": [
    { "id": 50, "full_name": "Adaora Okafor", ... },
    { "id": 51, "full_name": "Chidi Eze", ... }
  ]
}
```

Important: Extract owner IDs from response → `50` and `51`

Step 7: Upload Documents for Shareholder 1 (Adaora, ID: 50)

Upload ID document WITH owner_id:

```
POST /compliance/documents/uploads
Headers:
  Idempotency-Key: <uuid>

Body:
{
  "purpose": "shareholder_identity",
  "owner_id": 50,                          ← REQUIRED for shareholders!
  "file_name": "adaora-passport.pdf",
  "content_type": "application/pdf",
  "size_bytes": 512000
}

Response: { "document_id": "doc_sh1_id", ... }
```

Upload to S3, then complete:

```
POST /compliance/documents/doc_sh1_id/complete
```

Auto-Link! The backend AUTOMATICALLY sets:
`owner(50).id_document_id = "doc_sh1_id"`

Upload proof of address WITH owner_id:

```
POST /compliance/documents/uploads
{
  "purpose": "shareholder_address",
  "owner_id": 50,
  "file_name": "adaora-utility.pdf",
  ...
}

// Upload to S3, then complete
POST /compliance/documents/doc_sh1_addr/complete
```

> **Auto-Link!** Backend sets:
> `owner(50).proof_of_address_document_id = "doc_sh1_addr"`

Step 8: Upload Documents for Shareholder 2 (Chidi, ID: 51)

Same pattern as Step 7, but with `owner_id: 51`

```
POST /compliance/documents/uploads
{ "purpose": "shareholder_identity", "owner_id": 51, ... }
→ Complete → auto-links to owner 51

POST /compliance/documents/uploads
{ "purpose": "shareholder_address", "owner_id": 51, ... }
→ Complete → auto-links to owner 51
```

Step 9: Verify (Optional)

```
GET /compliance

Response shows owners_complete: true and all documents linked:
{
  "owners": [
    { "id": 50, "id_document_id": "doc_sh1_id",
"proof_of_address_document_id": "doc_sh1_addr" },
    { "id": 51, "id_document_id": "doc_sh2_id",
"proof_of_address_document_id": "doc_sh2_addr" }
```

```
  ]
}
```

Step 10-11: Save Bank Account & Submit

```
PUT /compliance/bank-accounts
{ "bank_accounts": [{ "bank_code": "058", "account_number":
"1234567890", "account_name": "TECHCORP LIMITED" }] }
→ { "version": 6 }

POST /compliance/submit
{ "agreement_accepted": true }
→ { "version": 7, "status": "submitted" }
```

> **Registered Business Flow Complete!**

9. Document Upload Flow

Three-Step Process

Every document upload follows these 3 steps:

Step	Endpoint	Description
1	`POST /compliance/documents/uploads`	Request presigned URL from backend
2	`PUT {upload_url}`	Upload file directly to S3 (not through API!)
3	`POST /compliance/documents/{id}/complete`	Confirm completion (triggers auto-link for shareholders)

Document Purpose Reference

Purpose	Business Type	Description	owner_id Required?
`identity`	Starter	Owner ID document	**NO**
`owner_address`	Starter	Owner proof of address	**NO**
`business_address`	Both	Business address proof	**NO**
`shareholder_identity`	Registered	Shareholder ID	**YES**
`shareholder_address`	Registered	Shareholder address	**YES**

Supported File Types

Content Type	Extensions	Max Size

`application/pdf`	.pdf	10 MB
`image/jpeg`	.jpg, .jpeg	10 MB
`image/png`	.png	10 MB

10. Error Handling

HTTP Status Codes

Code	Meaning	Action
200	Success	Process response
400	Bad Request	Fix request data
401	Unauthorized	Refresh token or re-login
409	Conflict	Version mismatch - refresh and retry
422	Validation Error	Fix field errors
500	Server Error	Retry with backoff

Error Response Format

```
{
  "success": false,
  "message": "Validation failed",
  "error_code": "VALIDATION_FAILED",
  "errors": [
    { "step": "owners", "field": "id_document_id", "message": "ID
document is required" }
  ]
}
```

Common Error Codes

Code	Description
`VALIDATION_FAILED`	Field validation errors
`VERSION_CONFLICT`	X-Expected-Version doesn't match
`IDEMPOTENCY_CONFLICT`	Same key, different request body
`DOCUMENT_NOT_FOUND`	Document ID doesn't exist
`OWNER_NOT_FOUND`	Owner ID doesn't exist
`ALREADY_SUBMITTED`	Cannot modify submitted compliance

© 2026 IRPay. All rights reserved.